NEW BATCH STARTING SOON | REVISED EDITION

# SQL • PYTHON • PYSPARK

## CODING INTERVIEW Q&A

45 Scenario-Based Questions | 3–8 Years Experience | Interview Focused

SQL Window Functions • Python Automation • PySpark Optimization • Delta Lake

Book Now to Get Early Bird Discount Offer
**https://topmate.io/asheesh/2145485**

# SQL — Scenario-Based Coding Questions

15 questions covering window functions, deduplication, gaps & islands, and query optimization.

### Q1. You have an orders table and suspect duplicate rows on order_id. How do you find and remove them, keeping only the latest record by updated_at?

**A.** Use ROW_NUMBER() partitioned by the natural key, ordered by the recency column, to identify every row that is not the latest. Delete by a unique row identifier rather than the duplicated business key, so only the extra rows are removed and the one you meant to keep survives.

```
-- SQL Server / any dialect supporting CTE deletes
WITH ranked AS (
SELECT *,
ROW_NUMBER() OVER (
PARTITION BY order_id
ORDER BY updated_at DESC
) AS rn
FROM orders
)
DELETE FROM ranked WHERE rn > 1;

-- PostgreSQL (no unique key needed, uses ctid)
DELETE FROM orders o
USING (
SELECT ctid,
ROW_NUMBER() OVER (
PARTITION BY order_id ORDER BY updated_at DESC
) AS rn
FROM orders
) r
WHERE o.ctid = r.ctid AND r.rn > 1;

-- MySQL (needs a real primary key, e.g. id)
DELETE o FROM orders o
JOIN (
SELECT id,
ROW_NUMBER() OVER (
PARTITION BY order_id ORDER BY updated_at DESC
) AS rn
FROM orders
) r ON o.id = r.id
WHERE r.rn > 1;
```

### Q2. Find the second highest salary in each department without using LIMIT/OFFSET.

**A.** DENSE_RANK() handles ties correctly, so filter for rank 2 per department.

```
SELECT department_id, employee_id, salary
FROM (
SELECT department_id, employee_id, salary,
DENSE_RANK() OVER (
PARTITION BY department_id
ORDER BY salary DESC
) AS rnk
FROM employees
) t
WHERE rnk = 2;
```

### Q3. Calculate a running total of daily sales per store using a window function.

**A.** SUM() as a window function with an ORDER BY clause gives a cumulative total without a self-join.

```
SELECT store_id, sale_date, daily_sales,
SUM(daily_sales) OVER (
PARTITION BY store_id
ORDER BY sale_date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS running_total
FROM daily_sales;
```

### Q4. List employees earning more than the average salary of their own department.

**A.** A correlated subquery or a window function both work; the window version avoids repeated scans.

```
SELECT employee_id, department_id, salary
FROM (
SELECT employee_id, department_id, salary,
AVG(salary) OVER (PARTITION BY department_id) AS dept_avg
FROM employees
) t
WHERE salary > dept_avg;
```

### Q5. A logins table has one row per user per day. Find users who logged in on 5+ consecutive days.

**A.** Subtract a row number from the date to create a constant group value for consecutive dates, then count group sizes. The date-arithmetic function differs by engine.

```
-- MySQL
WITH grp AS (
SELECT user_id, login_date,
DATE_SUB(login_date,
INTERVAL ROW_NUMBER() OVER (
PARTITION BY user_id ORDER BY login_date
) DAY
) AS streak_grp
FROM logins
)
SELECT user_id, MIN(login_date) AS streak_start, COUNT(*) AS streak_len
FROM grp
GROUP BY user_id, streak_grp
HAVING COUNT(*) >= 5;

-- PostgreSQL: login_date - (ROW_NUMBER() OVER (...) || ' days')::interval
-- SQL Server / Snowflake: DATEADD(day, -ROW_NUMBER() OVER (...), login_date)
```

### Q6. Return the top 3 products by revenue within every category.

**A.** ROW_NUMBER() partitioned by category lets you cut off at 3 per group in one pass.

```
SELECT category, product_id, revenue
FROM (
SELECT category, product_id, revenue,
ROW_NUMBER() OVER (
PARTITION BY category ORDER BY revenue DESC
) AS rnk
FROM product_revenue
) t
WHERE rnk <= 3;
```

### Q7. Pivot a long monthly_sales(product_id, month, amount) table into one row per product with a column per month.

**A.** Use conditional aggregation, which is portable across engines that do not support a native PIVOT.

```
SELECT product_id,
SUM(CASE WHEN month = 1 THEN amount END) AS jan,
SUM(CASE WHEN month = 2 THEN amount END) AS feb,
SUM(CASE WHEN month = 3 THEN amount END) AS mar
FROM monthly_sales
GROUP BY product_id;
```

### Q8. Find customers who made at least one purchase in every month of 2025.

**A.** Count distinct purchase months per customer and compare against the total number of months required.

```
SELECT customer_id
FROM orders
WHERE YEAR(order_date) = 2025
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(order_date)) = 12;
```

### Q9. Calculate month-over-month revenue growth percentage.

**A.** Compute the prior month's value once in a CTE, then reuse it, instead of calling LAG() twice in the same SELECT.

```
WITH prev AS (
SELECT month, revenue,
LAG(revenue) OVER (ORDER BY month) AS prev_revenue
FROM monthly_revenue
)
SELECT month, revenue,
ROUND((revenue - prev_revenue) / prev_revenue * 100, 2) AS mom_growth_pct
FROM prev;
```

### Q10. Group web events into sessions where a new session starts after a 30-minute gap of inactivity per user.

**A.** Compare each event's timestamp to the previous one, using COALESCE so the very first event per user does not evaluate the gap against NULL, then cumulative-sum the new-session flags.

```
WITH gaps AS (
SELECT user_id, event_time,
CASE WHEN TIMESTAMPDIFF(MINUTE,
COALESCE(
LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time),
event_time
),
event_time) > 30
THEN 1 ELSE 0 END AS new_session
FROM events
)
SELECT user_id, event_time,
SUM(new_session) OVER (
PARTITION BY user_id ORDER BY event_time
) AS session_id
FROM gaps;
```

### Q11. Write a recursive query to return the full management chain for every employee.

**A.** A recursive CTE anchors on top-level managers, then repeatedly joins children to parents to build the hierarchy.

```
WITH RECURSIVE org AS (
SELECT employee_id, manager_id, employee_id AS top_manager, 1 AS lvl
FROM employees WHERE manager_id IS NULL
UNION ALL
SELECT e.employee_id, e.manager_id, org.top_manager, org.lvl + 1
FROM employees e
JOIN org ON e.manager_id = org.employee_id
)
SELECT * FROM org ORDER BY top_manager, lvl;
```

### Q12. A report joining three large fact tables is timing out. How would you diagnose and speed it up?

**A.** Run EXPLAIN to see the execution plan, look for full table scans and nested-loop joins on unindexed columns, and check join order and cardinality estimates.

```
EXPLAIN ANALYZE
SELECT ...
FROM fact_sales f
JOIN fact_returns r ON f.order_id = r.order_id
JOIN dim_product p ON f.product_id = p.product_id;

-- From there: add indexes on join/filter columns, filter
-- earlier (push predicates before joins), pre-aggregate
-- where possible, and partition the largest fact table.
```

### Q13. Find date gaps in a table that should have one row per calendar day.

**A.** Generate a full date series and LEFT JOIN it against the actual table to expose missing dates.

```
WITH RECURSIVE calendar AS (
SELECT MIN(txn_date) AS d FROM daily_txn
UNION ALL
SELECT DATE_ADD(d, INTERVAL 1 DAY) FROM calendar
WHERE d < (SELECT MAX(txn_date) FROM daily_txn)
)
SELECT c.d AS missing_date
FROM calendar c
LEFT JOIN daily_txn t ON c.d = t.txn_date
WHERE t.txn_date IS NULL;
```

### Q14. Identify customers who have churned: no purchase in the last 90 days, but purchased before that.

**A.** Aggregate the most recent and earliest order date per customer, then require the last order to be more than 90 days old AND an earlier order to exist before that 90-day cutoff, so a brand-new one-time buyer does not get flagged as churned.

```
SELECT customer_id
FROM orders
GROUP BY customer_id
HAVING MAX(order_date) < CURRENT_DATE - INTERVAL '90 days'
AND MIN(order_date) < CURRENT_DATE - INTERVAL '90 days';
```

### Q15. You need to compare each row's value to the average of the previous 3 rows (a moving average) for anomaly detection.

**A.** A window frame of ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING excludes the current row so it is not averaging itself.

```
SELECT sensor_id, reading_time, value,
AVG(value) OVER (
PARTITION BY sensor_id
ORDER BY reading_time
ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING
) AS prev_3_avg
FROM sensor_readings;

-- Flag rows where value deviates from prev_3_avg by more
-- than a chosen threshold as anomalies.
```

# Python — Scenario-Based Coding Questions

15 questions covering data structures, streaming/batching, concurrency, and clean production patterns.

### Q1. Given a large log file that does not fit in memory, count occurrences of each error type efficiently.

**A.** Stream the file line by line instead of loading it fully, and use a Counter to tally matches incrementally.

```
from collections import Counter

def count_errors(path):
counts = Counter()
with open(path) as f:
for line in f:
if "ERROR" in line:
err_type = line.split("ERROR")[1].split(":")[0].strip()
counts[err_type] += 1
return counts
```

### Q2. You receive a list of (start, end) meeting intervals and need to merge all overlapping ones.

**A.** Sort by start time, then walk through and merge whenever the current interval overlaps the last merged one.

```
def merge_intervals(intervals):
intervals.sort(key=lambda x: x[0])
merged = [intervals[0]]
for start, end in intervals[1:]:
last_start, last_end = merged[-1]
if start <= last_end:
merged[-1] = (last_start, max(last_end, end))
else:
merged.append((start, end))
return merged
```

### Q3. Find the missing number in a list containing n distinct numbers from 0 to n.

**A.** Use the closed-form sum of 0..n and subtract the actual sum, giving O(n) time and O(1) space.

```
def find_missing(nums):
n = len(nums)
expected_sum = n * (n + 1) // 2
return expected_sum - sum(nums)
```

### Q4. You need to deduplicate a stream of order records, keeping only the latest by timestamp for each order_id.

**A.** A dict keyed by order_id naturally keeps the last-seen (or explicitly latest) record as you iterate.

```
def dedupe_latest(records):
latest = {}
for r in records:
oid = r["order_id"]
if oid not in latest or r["timestamp"] > latest[oid]["timestamp"]:
latest[oid] = r
return list(latest.values())
```

### Q5. Flatten an arbitrarily nested list, e.g. [1, [2, [3, 4], 5], 6] -> [1, 2, 3, 4, 5, 6].

**A.** A recursive generator handles any nesting depth cleanly.

```
def flatten(nested):
for item in nested:
if isinstance(item, list):
yield from flatten(item)
else:
yield item

result = list(flatten([1, [2, [3, 4], 5], 6]))
```

### Q6. Implement an LRU cache from scratch without using functools.lru_cache.

**A.** OrderedDict gives O(1) access while preserving insertion/usage order for eviction.

```
from collections import OrderedDict

class LRUCache:
def __init__(self, capacity):
self.capacity = capacity
self.cache = OrderedDict()

def get(self, key):
if key not in self.cache:
return -1
self.cache.move_to_end(key)
return self.cache[key]

def put(self, key, value):
if key in self.cache:
self.cache.move_to_end(key)
self.cache[key] = value
if len(self.cache) > self.capacity:
self.cache.popitem(last=False)
```

### Q7. Given a list of integers, find the length of the longest run of consecutive numbers (order does not matter).

**A.** Put values in a set for O(1) lookups, then only start counting a sequence from numbers that are a true start, meaning n-1 is not in the set.

```
def longest_consecutive(nums):
num_set = set(nums)
best = 0
for n in num_set:
if n - 1 not in num_set:
length = 1
while n + length in num_set:
length += 1
best = max(best, length)
return best
```

### Q8. Group a list of words into anagram clusters.

**A.** Sorted letters make a canonical key that is identical for all anagrams of the same word.

```
from collections import defaultdict

def group_anagrams(words):
groups = defaultdict(list)
for w in words:
key = "".join(sorted(w))
groups[key].append(w)
return list(groups.values())
```

### Q9. An API you call occasionally times out or rate-limits. Write a reusable retry mechanism with exponential backoff.

**A.** A decorator keeps retry logic separate from business logic and can be reused across every API call. Add jitter and a max delay cap so retries do not synchronize into further bursts, and catch only the transient exceptions you expect rather than every Exception.

```
import time, random, functools

def retry_with_backoff(retries=5, base_delay=1, max_delay=30,
exceptions=(ConnectionError, TimeoutError)):
def decorator(func):
@functools.wraps(func)
def wrapper(*args, **kwargs):
delay = base_delay
for attempt in range(retries):
try:
return func(*args, **kwargs)
except exceptions:
if attempt == retries - 1:
raise
time.sleep(delay + random.uniform(0, delay * 0.1))
delay = min(delay * 2, max_delay)
return wrapper
return decorator
```

### Q10. You are consuming Kafka messages and must insert them into a database in batches of 100 for efficiency.

**A.** A generator buffers incoming events and yields a full batch once it reaches the threshold, clearing the buffer after each yield.

```
def batch_events(stream, batch_size=100):
batch = []
for event in stream:
batch.append(event)
if len(batch) == batch_size:
yield batch
batch = []
if batch:
yield batch

for batch in batch_events(kafka_stream):
insert_into_db(batch)
```

### Q11. You need to pull data from 20 REST endpoints as fast as possible without overwhelming the source system.

**A.** A thread pool with a bounded number of workers parallelizes I/O-bound calls while capping concurrency.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import requests

def fetch(url):
return requests.get(url, timeout=10).json()

def fetch_all(urls, max_workers=5):
results = []
with ThreadPoolExecutor(max_workers=max_workers) as pool:
futures = {pool.submit(fetch, u): u for u in urls}
for future in as_completed(futures):
results.append(future.result())
return results
```

### Q12. Write a custom context manager that guarantees a database connection is closed even if an exception occurs mid-query.

**A.** The contextlib.contextmanager decorator makes this concise with a try/finally around the yield.

```
from contextlib import contextmanager

@contextmanager
def db_connection(conn_string):
conn = create_connection(conn_string)
try:
yield conn
finally:
conn.close()

with db_connection(CONN_STR) as conn:
conn.execute("SELECT * FROM orders")
```

### Q13. Given a list of numeric sensor readings, flag values that are statistical outliers.

**A.** The IQR (interquartile range) method flags anything far outside the middle 50% of the data without assuming a normal distribution.

```
import statistics as stats

def find_outliers(values):
sorted_vals = sorted(values)
q1 = stats.median(sorted_vals[:len(sorted_vals)//2])
q3 = stats.median(sorted_vals[(len(sorted_vals)+1)//2:])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
return [v for v in values if v < lower or v > upper]
```

### Q14. Validate a large list of email addresses and return only the valid ones for a data quality check.

**A.** A compiled regex avoids re-parsing the pattern on every row when checking large datasets.

```
import re

EMAIL_RE = re.compile(r"^[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}$")

def valid_emails(emails):
return [e for e in emails if EMAIL_RE.match(e)]
```

### Q15. Implement a simple rate limiter that allows at most N API calls per rolling 60-second window.

**A.** A deque of call timestamps lets you evict entries older than the window and check remaining capacity in amortized O(1) time.

```
from collections import deque
import time

class RateLimiter:
def __init__(self, max_calls, window_seconds=60):
self.max_calls = max_calls
self.window = window_seconds
self.calls = deque()

def allow(self):
now = time.time()
while self.calls and now - self.calls[0] > self.window:
self.calls.popleft()
if len(self.calls) < self.max_calls:
self.calls.append(now)
return True
return False
```

# PySpark — Scenario-Based Coding Questions

15 questions covering skew, joins, Delta Lake, streaming, and performance tuning.

### Q1. You are reading a CSV where some rows are malformed and crash the job. How do you make ingestion resilient?

**A.** Use PERMISSIVE mode with a badRecordsPath (or a _corrupt_record column) so bad rows are quarantined instead of failing the whole job.

```
df = (
spark.read
.option("header", True)
.option("mode", "PERMISSIVE")
.option("badRecordsPath", "/mnt/quarantine/orders")
.option("columnNameOfCorruptRecord", "_corrupt_record")
.csv("/mnt/raw/orders/")
)
clean_df = df.filter(df["_corrupt_record"].isNull())
```

### Q2. A Delta table has duplicate rows per business key. Deduplicate it, keeping only the most recent record.

**A.** row_number() over a window partitioned by the key and ordered by recency isolates the latest row per key.

```
from pyspark.sql import Window
from pyspark.sql.functions import row_number, col

w = Window.partitionBy("order_id").orderBy(col("updated_at").desc())
deduped = (
df.withColumn("rn", row_number().over(w))
.filter(col("rn") == 1)
.drop("rn")
)
```

### Q3. A join between two large tables is extremely slow because one join key is heavily skewed. How do you fix it?

**A.** Salt the skewed key on both sides so the hot key's rows are spread across multiple partitions instead of overloading one executor.

```
from pyspark.sql.functions import concat, lit, rand, floor, explode, array, col

N = 10 # number of salt buckets
left_salted = left_df.withColumn(
"salt", floor(rand() * N)
).withColumn("join_key", concat(col("key"), lit("_"), col("salt")))

right_salted = right_df.withColumn(
"salt", explode(array([lit(i) for i in range(N)]))
).withColumn("join_key", concat(col("key"), lit("_"), col("salt")))

result = left_salted.join(right_salted, "join_key")
```

### Q4. You are joining a 500 GB fact table with a 2 MB lookup table and the shuffle is unnecessarily expensive.

**A.** Broadcast the small table so Spark sends a copy to every executor and avoids a shuffle join entirely.

```
from pyspark.sql.functions import broadcast

result = fact_df.join(broadcast(lookup_df), "product_id")
```

### Q5. Design an incremental load that merges new/changed records into a large Delta table without reprocessing everything.

**A.** Delta's MERGE INTO does an upsert in one atomic operation using a watermark or CDC column to scope the incoming batch.

```
from delta.tables import DeltaTable

target = DeltaTable.forPath(spark, "/mnt/delta/customers")
(
target.alias("t")
.merge(source_df.alias("s"), "t.customer_id = s.customer_id")
.whenMatchedUpdateAll()
.whenNotMatchedInsertAll()
.execute()
)
```

### Q6. You receive deeply nested JSON with structs and arrays and need a flat table for BI tools.

**A.** Use dot notation to pull out struct fields and explode() to unnest arrays into rows.

```
from pyspark.sql.functions import explode, col

flat = (
df.select(
"order_id",
col("customer.name").alias("customer_name"),
col("customer.address.city").alias("city"),
explode("line_items").alias("item")
)
.select("order_id", "customer_name", "city", "item.sku", "item.qty")
)
```

### Q7. Turn a long sales(product_id, month, amount) DataFrame into a wide table with one column per month.

**A.** groupBy().pivot() is Spark's native equivalent of a SQL PIVOT.

```
pivoted = (
sales_df.groupBy("product_id")
.pivot("month")
.sum("amount")
)
```

### Q8. Rank products by revenue within each category and compute a running total per category.

**A.** Window functions in PySpark mirror SQL; define a WindowSpec and apply rank()/sum() over it.

```
from pyspark.sql import Window
from pyspark.sql.functions import rank, sum as _sum, col

w = Window.partitionBy("category").orderBy(col("revenue").desc())
w_running = (
Window.partitionBy("category").orderBy("revenue")
.rowsBetween(Window.unboundedPreceding, Window.currentRow)
)

result = (
df.withColumn("rank", rank().over(w))
.withColumn("running_total", _sum("revenue").over(w_running))
)
```

### Q9. An upstream API feed changes its JSON schema often (new/renamed fields). How do you keep the pipeline from breaking?

**A.** Auto Loader's schema inference and evolution tracks schema changes automatically and lets new columns merge in rather than fail the job.

```
df = (
spark.readStream
.format("cloudFiles")
.option("cloudFiles.format", "json")
.option("cloudFiles.schemaLocation", "/mnt/schema/events")
.option("cloudFiles.schemaEvolutionMode", "addNewColumns")
.load("/mnt/raw/events/")
)
```

### Q10. A job produces way too many small output files and downstream reads are slow. How do you fix the partitioning?

**A.** Use coalesce() to reduce partitions without a full shuffle when writing, and repartition() when you actually need to redistribute or increase parallelism upstream.

```
# Reduce output file count before writing (no shuffle)
df.coalesce(8).write.format("delta").mode("overwrite").save("/mnt/delta/sales")

# Redistribute data evenly before a wide transformation (shuffle)
df2 = df.repartition(200, "customer_id")
```

### Q11. A slow-running job is heavily using a Python UDF for a simple string transformation. How do you speed it up?

**A.** Python UDFs serialize row by row between the JVM and Python, which is expensive. Replace with Spark's built-in native functions whenever possible.

```
# Slow: Python UDF
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

upper_udf = udf(lambda s: s.upper() if s else None, StringType())
df = df.withColumn("name_upper", upper_udf("name"))

# Fast: native Spark function, no serialization overhead
from pyspark.sql.functions import upper
df = df.withColumn("name_upper", upper("name"))
```

### Q12. You need to track full history of changes to a customer dimension (SCD Type 2) in Delta Lake.

**A.** MERGE INTO with conditional update/insert clauses can close out the old record and insert a new versioned row in a single transaction.

```
from delta.tables import DeltaTable

target = DeltaTable.forPath(spark, "/mnt/delta/dim_customer")
(
target.alias("t")
.merge(updates_df.alias("s"),
"t.customer_id = s.customer_id AND t.is_current = true")
.whenMatchedUpdate(set={
"is_current": "false",
"end_date": "current_date()"
})
.execute()
)

new_rows = (
updates_df.withColumn("is_current", lit(True))
.withColumn("start_date", current_date())
)
new_rows.write.format("delta").mode("append").save("/mnt/delta/dim_customer")
```

### Q13. Design a streaming pipeline that reads from Kafka and writes to a Delta table with exactly-once guarantees.

**A.** Structured Streaming with a checkpoint location tracks offsets so restarts resume exactly where they left off, and Delta writes are transactional.

```
kafka_df = (
spark.readStream.format("kafka")
.option("kafka.bootstrap.servers", KAFKA_BROKERS)
.option("subscribe", "orders")
.load()
)

parsed_df = (
kafka_df.selectExpr("CAST(value AS STRING) as json")
.select(from_json("json", order_schema).alias("data"))
.select("data.*")
)

(
parsed_df.writeStream
.format("delta")
.option("checkpointLocation", "/mnt/checkpoints/orders")
.outputMode("append")
.start("/mnt/delta/orders")
)
```

### Q14. A DataFrame is reused across five downstream transformations and each one is recomputing it from scratch. How do you fix this?

**A.** cache() or persist() materializes the DataFrame after its first computation so subsequent actions reuse it instead of re-triggering the full lineage. Call unpersist() once it is no longer needed to free executor memory.

```
df = spark.read.parquet("/mnt/raw/large_dataset").filter("status = 'active'")
df.cache()
df.count() # triggers caching

summary1 = df.groupBy("region").count()
summary2 = df.groupBy("category").sum("revenue")
# both reuse the cached DataFrame

df.unpersist()
```

### Q15. In a streaming aggregation, some events arrive up to 20 minutes late. How do you handle them without the state growing unbounded?

**A.** withWatermark() tells Spark how long to wait for late data before finalizing and dropping state for a given window.

```
from pyspark.sql.functions import window

agg_df = (
events_df
.withWatermark("event_time", "20 minutes")
.groupBy(window("event_time", "10 minutes"), "device_id")
.count()
)

(
agg_df.writeStream
.outputMode("append")
.format("delta")
.option("checkpointLocation", "/mnt/checkpoints/device_counts")
.start("/mnt/delta/device_counts")
)
```

# Also Review Before Your Interview

For a 3–8 year Data Engineer interview, be ready to discuss these topics beyond the coding questions above.

## SQL

- Window frame types and defaults (RANGE vs ROWS)
- EXISTS vs IN vs JOIN, and when each is faster
- CROSS APPLY / LATERAL joins
- CTE vs temp table vs materialized view
- Reading and interpreting a query execution plan

## Python

- Multiprocessing vs multithreading and when to use each
- Generators vs iterators
- How decorators work under the hood
- Memory profiling for large in-memory jobs
- The GIL and its practical impact on CPU-bound code

## PySpark

- Narrow vs wide transformations
- Catalyst optimizer and Tungsten execution engine
- Adaptive Query Execution (AQE), including AQE skew join handling
- optimizeWrite and autoCompact on Delta tables
- Partition pruning and dynamic partition overwrite
- Broadcast hash join vs sort-merge join
- Shuffle internals and how to reduce shuffle cost

Want to go deeper on these with hands-on, end-to-end pipeline projects?
Join the live Azure Data Engineering bootcamp.
Book now for the early bird discount at
**https://topmate.io/asheesh/2145485**